

Teoria Współbieżności

Laboratorium 10 Część II

Teoria śladów

Kacper Bieniasz

16 grudnia 2024

1 Zadanie

Napisz program w dowolnym języku, który:

1. Wyznacza relację zależności D
2. Wyznacza ślad $[w]$ względem relacji I i D
3. Wyznacza postać normalną Foaty $FNF([w])$ śladu $[w]$
4. Wyznacza graf zależności dla słowa w
5. Wyznacza postać normalną Foaty na podstawie grafu

Dane wejściowe programu:

- Alfabet A , w którym każda litera oznacza akcję.
- Relacja niezależności I , oznaczająca które akcje są niezależne (przemienne, tzn. można je wykonać w dowolnej kolejności i nie zmienia to wyniku końcowego).
- Słowo w oznaczające przykładowe wykonanie sekwencji akcji.

Do wyznaczenia postaci normalnej Foaty wykorzystać algorytm z pracy Volker Diekert, Yves Métivier: Partial Commutation and Traces str.11.

2 Implementacja zadania

Jako język do implementacji wybrałem język Python w wersji 3.12. Wykorzystane biblioteki:

- networkx - służy do reprezentacji grafów i wykonywania na nich operacji
- matplotlib.pyplot - pozwala rysować wykresy, struktury danych, grafy itp.
- collections - zawiera implementacje struktur danych między innymi kolejki priorytetowej

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 from collections import deque
```

Kod Python 1: Import wykorzystywanych bibliotek i struktur danych

Kod (2) przedstawia funkcję wyznaczającą relację zależności D . Jako argumenty funkcja przyjmuje alfabet A oraz zbiór relacji niezależności I . Funkcja wykorzystuje informację, że D jest dopełnieniem I . Matematycznie możemy zapisać powyższą relację jako: $D = (A \times A) \setminus I$.

```

1 def compute_dependency_relation(A, I):
2     D = set()
3     for x in A:
4         for y in A:
5             if (x, y) not in I:
6                 D.add((x, y))
7     return D

```

Kod Python 2: Funkcja wyznaczająca relację zależności

Wyznaczenie ślady słowa w realizuje funkcja $compute_trace(w, I)$. Przyjmuje ona słowo w oraz zbiór zdarzeń niezależnych I . Kod tej funkcji zawiera blok (3).

```

1 def compute_trace(w, I):
2     visited = set()
3     queue = deque()
4     visited.add(w)
5     queue.append(w)
6     while queue:
7         current = queue.popleft()
8         for i in range(len(current) - 1):
9             x, y = current[i], current[i + 1]
10            if (x, y) in I:
11                new_word = current[:i] + y + x + current[i + 2:]
12                if new_word not in visited:
13                    visited.add(new_word)
14                    queue.append(new_word)
15    return visited

```

Kod Python 3: Funkcja wyznaczająca ślad słowa w

Wyznaczenie postaci normalnej Foaty według zaproponowanego algorytmu, podzielimy na etapy. Pierwszy zajmie się stworzeniem stosów dla liter ze słowa w . Kod funkcji realizujących to zadanie przedstawia blok (4).

```

1 def build_stacks_for_word(w, A, I):
2     stacks = {x: [] for x in A}
3     for letter in reversed(w):
4         stacks[letter].append(letter)
5         for b in A:
6             if b != letter and do_not_commute(letter, b, I):
7                 stacks[b].append(None)
8     return stacks
9 def do_not_commute(x, y, I):
10    return (x, y) not in I

```

Kod Python 4: Funkcje wykorzystywane do zbudowania stosów dla każdej litery ze słowa w

Następnie zajmiemy się ściąganiem ze stosów liter i odpowiadającym im markerom. Funkcja realizująca ten etap zwraca listę warstw, gdzie każda warstwa to zbiór liter. Jej kod przedstawia blok (5).

```

1 def fnf_from_stacks(stacks, A, I):
2     foata_layers = []
3     while any(stacks[x] for x in A):
4         top_letters = []
5         for x in A:
6             if stacks[x]:
7                 top_element = stacks[x][-1]
8                 if top_element == x:
9                     top_letters.append(x)

```

```

10     top_letters.sort()
11     foata_layers.append(top_letters)
12     for a in top_letters:
13         stacks[a].pop()
14         for b in A:
15             if b != a and do_not_commute(a, b, I):
16                 if stacks[b]:
17                     topb = stacks[b][-1]
18                     if topb is None:
19                         stacks[b].pop()
20     return foata_layers

```

Kod Python 5: Funkcja tworząca warstwy wykorzystywane do budowy postaci normalnej Foaty śladu słowa w

Ostatnim etapem będzie zamiana warstw na postać normalną Foaty śladu słowa w . Kod funkcji realizującej to zadanie zawiera blok (6).

```

1 def fnf_to_string(layers):
2     return ''.join('(' + ''.join(block) + ')') for block in layers)

```

Kod Python 6: Funkcja zamiany warstw na postać normalną Foaty śladu słowa w

Następnie zajmiemy się zbudowaniem grafu zależności Dickerta. Kod Funkcji wyznaczającej graf w postaci minimalnej przedstawia blok (7).

```

1 def build_dependency_graph(w, I):
2     n = len(w)
3     graph = {i: [] for i in range(n)}
4     for i in range(n):
5         for j in range(i + 1, n):
6             if (w[i], w[j]) not in I:
7                 graph[i].append(j)
8     def can_reach(u, v, adj):
9         visited = set([u])
10        queue = deque([u])
11        while queue:
12            curr = queue.popleft()
13            if curr == v:
14                return True
15            for nxt in adj[curr]:
16                if nxt not in visited:
17                    visited.add(nxt)
18                    queue.append(nxt)
19        return False
20    for u in range(n):
21        neighbors_copy = list(graph[u])
22        for v in neighbors_copy:
23            graph[u].remove(v)
24            if not can_reach(u, v, graph):
25                graph[u].append(v)
26    return graph

```

Kod Python 7: Funkcja wyznaczająca graf zależności dla słowa w

Dodatkowo dobrze byłoby wyświetlić graf, a nie bazować tylko na jego reprezentacji w postaci listy sąsiedztwa. Funkcja zajmująca się wyświetleniem grafu zależności przedstawia blok (8).

```

1 def draw_dependency_graph_networkx(w, adjacency_list):
2     G = nx.DiGraph()
3     for i in range(len(w)):
4         G.add_node(i, label=f"{i}:{w[i]}")
5     for i, neighbors in adjacency_list.items():
6         for j in neighbors:
7             G.add_edge(i, j)
8     pos = nx.circular_layout(G)
9     nodes_collection = nx.draw_networkx_nodes(G, pos, node_color="
10         lightblue", node_size=1200)
11     edges_collection = nx.draw_networkx_edges(G, pos, arrows=True,
12         arrowstyle='->', arrowsize=20)
13     nodes_collection.set_zorder(1)
14     for edge_item in edges_collection:
15         edge_item.set_zorder(2)
16     labels = {node: data["label"] for node, data in G.nodes(data=True)}
17     nx.draw_networkx_labels(G, pos, labels, font_size=10)
18     plt.title("Dependency Graph")
19     plt.axis("off")
20     plt.show()

```

Kod Python 8: Funkcja wyświetlająca graf zależności

Do implementacji została nam ostatnia funkcjonalność, czyli wyznaczenie postaci normalnej Foaty na podstawie grafu. Schemat postępowania wygląda podobnie jak w przypadku wyznaczenia postaci normalnej według algorytmu z pracy Partial Commutation and Traces. Najpierw wyznaczamy warstwy, a później przekształcamy do postaci normalnej. Kod funkcji realizujących to zadanie przedstawia blok (9).

```

1 def foata_normal_form_from_graph(w, graph):
2     n = len(w)
3     predecessors = {i: set() for i in range(n)}
4     for i in range(n):
5         for j in graph[i]:
6             predecessors[j].add(i)
7     remaining = set(range(n))
8     foata_layers = []
9     while remaining:
10         current_layer = [v for v in remaining if len(predecessors[v])
11             == 0]
12         foata_layers.append(current_layer)
13         for v in current_layer:
14             remaining.remove(v)
15             if v in graph:
16                 for w_next in graph[v]:
17                     predecessors[w_next].discard(v)
18     return foata_layers
19
20 def layers_to_FNF_string(w, foata_layers):
21     fnf_blocks = []
22     for layer in foata_layers:
23         layer_sorted = sorted(layer)
24         block_letters = ''.join(w[idx] for idx in layer_sorted)
25         fnf_blocks.append(f"({block_letters})")

```

```
25 return ''.join(fnf_blocks)
```

Kod Python 9: Funkcję wyznaczającą postać normalną Foaty na podstawie grafu

Brakuje jeszcze fragmentu wykonującego zrealizowane funkcjonalności. Blok (10) zawiera fragment kodu opowiedzianego za wykonanie funkcji operujących na słowie w oraz relacji niezależności I .

```
1 if __name__ == "__main__":
2     A = ...
3     I = ...
4     w = ...
5     print("Alfabet␣A:", A)
6     print("Relacja␣niezaleznosci␣I:", I)
7     print("Slovo␣w:", w)
8
9     D = compute_dependency_relation(A, I)
10    print("Relacja␣zaleznosci␣D:", D)
11
12    trace = compute_trace(w, I)
13    print("Slad␣[w]:", trace)
14
15    stacks = build_stacks_for_word(w, A, I)
16    layers = foata_normal_form_from_stacks(stacks, A, I)
17    fnf_str = fnf_to_string(layers)
18    print("Warstwy␣Foaty:", layers)
19    print("FNF([w])␣=", fnf_str)
20
21    graph = build_minimal_dependency_graph(w, I)
22    print("Graf␣zaleznosci␣(lista␣sasiedztwa):")
23    for k, v in graph.items():
24        print(f"␣␣{k}␣->␣{v}␣(litera␣'{w[k]}')")
25
26    foata_layers = foata_normal_form_from_graph(w, graph)
27    fnf_string = layers_to_FNF_string(w, foata_layers)
28    print("FNF([w])␣=", fnf_string)
29
30    draw_dependency_graph_networkx(w, graph)
```

Kod Python 10: Wykonanie programu

3 Otrzymane wyniki

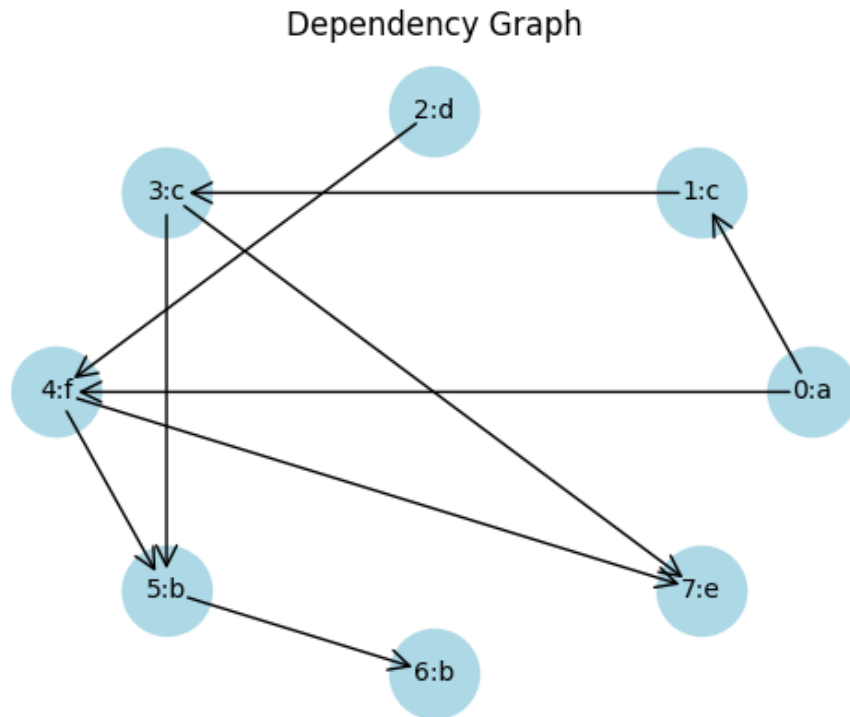
Dla danych testowych:

- $A = ['a', 'b', 'c', 'd']$
- $I = \{('a', 'd'), ('d', 'a'), ('b', 'c'), ('c', 'b')\}$
- $w = "baadcb"$

Otrzymane wyniki prezentują się następująco:

```

graph TD
    1a((1:a)) --> 2a((2:a))
    1a --> 3d((3:d))
    1a --> 5b((5:b))
    2a --> 4c((4:c))
    3d --> 4c
    3d --> 5b
  
```



Rysunek 2: Graf zależności

4 Wnioski

Stworzenie programu wyznaczającego relację zależności i niezależności, śladu słowa w , postaci normalnej Foaty śladu słowa w oraz grafu zależności pozwala w znacznie szybciej wyznaczać te właściwości w porównaniu z wykonywaniem takich działań na kartce. Dodatkowo przy skomplikowanych systemach współbieżnych wykonanie obliczeń samemu było by zbyt skomplikowane. Porównując otrzymane wyniki z wynikami części I laboratorium 10 zrobionymi na kartce program dostarcza takich wyników.

Literatura

- [1] https://www.researchgate.net/publication/280851316_Partial_Commutation_and_Traces
- [2] https://www.mimuw.edu.pl/~sl/teaching/22_23/TW/
- [3] https://www.researchgate.net/publication/272946925_Fast_GPU_integration_algorithm_for_isogeometric_finite_element_method_solvers_using_task_dependency_graphs